43 WHERE ARE THE SQL QUERIES? — HOW TYPEORM WRITES SQL FOR YOU

Look through src/student-pg/ and you won't find a single "SELECT ... FROM students" string — yet every request hits Postgres with real SQL. This file explains where that SQL comes from, how to SEE it, and the three levels of writing queries yourself when the auto-generated ones aren't enough.

Short answer: TypeORM GENERATES the SQL at runtime from your repository method calls (student-pg.service.ts) and your entity class (student.entity.ts), then hands it to the `pg` driver.

SECTION 1 — THE PIPELINE: METHOD CALL → SQL → POSTGRES

```
this.studentRepo.findOneBy({ id: 3 })          your code (service)
        |
        v
TypeORM reads the Student entity metadata       which table? which columns?
(@Entity('students'), @Column, ...)              (student.entity.ts)
        |
        v
TypeORM builds SQL text + parameters            SELECT ... WHERE "id" = $1
                                                params: [3]
        |
        v
pg driver sends it over TCP/SSL                 to Supabase / local Postgres
        |
        v
Postgres runs it, returns rows
        |
        v
TypeORM maps each row -> a Student object       plain JSON-able entity
        |
        v
service returns it -> controller -> HTTP response
```

The ENTITY is the map ("property name → column, class → table"). The REPOSITORY call is the intent ("find one by id"). TypeORM combines both into SQL. That's what an ORM (Object-Relational Mapper) is: objects in code <→ rows in tables.

SECTION 2 — EVERY QUERY IN THIS PROJECT

```
student-pg.service.ts               SQL generated

create()
  this.studentRepo.create(dto)      (none — builds an object in memory)
  this.studentRepo.save(student)    INSERT INTO "students"
("name","age","email","course","createdAt","updatedAt")
                                    VALUES ($1, $2, $3, $4, DEFAULT, DEFAULT)
                                    RETURNING "id", "createdAt", "updatedAt"

findAll()
  this.studentRepo.find()           SELECT "id","name","age","email","course","createdAt","updatedAt"
```

```
                                    FROM "students"

findOne(id)
  this.studentRepo.findOneBy({id})  SELECT ... FROM "students" WHERE "id" = $1 LIMIT 1

update(id, dto)
  this.studentRepo.preload({id,..}) SELECT ... FROM "students" WHERE "id" = $1
  this.studentRepo.save(student)    SELECT ... WHERE "id" = $1      (checks what changed)
                                    UPDATE "students" SET "age" = $1, "updatedAt" = CURRENT_TIMESTAMP
                                    WHERE "id" = $2

remove(id)
  this.findOne(id)                  SELECT ... WHERE "id" = $1 LIMIT 1
  this.studentRepo.delete(id)       DELETE FROM "students" WHERE "id" IN ($1)
```

Notes:

- save() does an INSERT when the object has no id, and an UPDATE when it has one — one method, two possible SQL statements.
- save() only puts CHANGED columns in the UPDATE.
- one service method can run several queries (update = 2-3).

SECTION 3 — EVEN THE TABLE WAS GENERATED (synchronize)

```
app.module.ts has  synchronize: true.  At every boot TypeORM:
  1. reads all entities (student.entity.ts)
  2. reads the real database structure (information_schema)
  3. runs DDL to make the DB match the entities:

  CREATE TABLE "students" (
    "id" SERIAL NOT NULL,
    "name" character varying NOT NULL,
    "age" integer NOT NULL,
    "email" character varying NOT NULL,
    "course" character varying,
    "createdAt" TIMESTAMP NOT NULL DEFAULT now(),
    "updatedAt" TIMESTAMP NOT NULL DEFAULT now(),
    CONSTRAINT "UQ_..." UNIQUE ("email"),
    CONSTRAINT "PK_..." PRIMARY KEY ("id")
  )

Add a property to the entity -> next boot runs ALTER TABLE ... ADD COLUMN.
Rename a property -> it may DROP the old column (data lost!) — why
synchronize is dev-only; production uses migrations, which write this
DDL into files you review first (theory/40 STEP 12).

DDL = Data Definition Language (CREATE / ALTER / DROP — structure)
DML = Data Manipulation Language (SELECT / INSERT / UPDATE / DELETE — data)
Section 2 is DML, this section is DDL.
```

SECTION 4 — HOW TO SEE THE GENERATED SQL

a) TypeORM logging (easiest) — app.module.ts: logging: true, // log every query logging: ['query', 'error'], // only queries + errors logging: ['error'], // only failures maxQueryExecutionTime: 500, // log queries slower than

500ms

Console output:

```
query: SELECT "Student"."id" AS "Student_id", ... FROM "students" "Student"
       WHERE (("Student"."id" = $1)) LIMIT 1 -- PARAMETERS: [3]
```

b) Print a query without running it (QueryBuilder): const qb = this.studentRepo.createQueryBuilder('s').where('s.id = :id', { id: 3 }); console.log(qb.getQueryAndParameters()); // ['SELECT ... FROM "students" "s" WHERE "s"."id" = $1', [3]]

c) Supabase dashboard → Database → Query Performance / Logs shows the queries the server actually received.

Why the aliases ("Student"."id" AS "Student_id")? TypeORM names every column uniquely so it can still map rows correctly when you JOIN several tables that both have an "id".

SECTION 5 — LEVEL 1: REPOSITORY METHODS WITH OPTIONS (still no SQL)

Most real queries are still just repository calls with options:

```
import { MoreThan, LessThan, Between, Like, ILike, In, IsNull, Not } from 'typeorm';

this.studentRepo.find({
  where: { age: MoreThan(20), course: 'CS' },   // WHERE age > $1 AND course = $2
  order: { name: 'ASC' },                       // ORDER BY name ASC
  skip: 0, take: 10,                            // OFFSET 0 LIMIT 10 (pagination)
  select: { id: true, name: true },             // SELECT id, name
});
```

Operator → SQL:

MoreThan(20)	age > 20	
LessThan(30)	age < 30	
Between(18, 25)	age BETWEEN 18 AND 25	
Like('%sharma%')	name LIKE '%sharma%'	(case-sensitive)
ILike('%sharma%')	name ILIKE '%sharma%'	(case-insensitive, Postgres)
In([2, 3])	id IN (2, 3)	
IsNull()	course IS NULL	
Not('CS')	course != 'CS'	

```
OR conditions = an ARRAY of where objects:
  where: [{ course: 'CS' }, { age: MoreThan(25) }]   // course='CS' OR age>25
```

Other handy methods:

```
this.studentRepo.count({ where: { course: 'CS' } })   // SELECT COUNT(*)
this.studentRepo.exists({ where: { email } })          // SELECT 1 ... LIMIT 1
this.studentRepo.findAndCount({ take: 10 })            // rows + total (pagination)
```

```
this.studentRepo.update(id, { age: 23 })              // UPDATE without loading first
this.studentRepo.increment({ id }, 'age', 1)          // SET age = age + 1
```

SECTION 6 — LEVEL 2: QUERY BUILDER (SQL-shaped, still safe)

When options aren't enough (GROUP BY, complex joins, sub-queries):

```
// students per course
this.studentRepo
  .createQueryBuilder('s')                    // 's' = alias for students
  .select('s.course', 'course')
  .addSelect('COUNT(*)', 'total')
  .where('s.age > :minAge', { minAge: 18 })   // :minAge = named parameter
  .groupBy('s.course')
  .orderBy('total', 'DESC')
  .getRawMany();

-> SELECT "s"."course" AS "course", COUNT(*) AS "total"
   FROM "students" "s" WHERE "s"."age" > $1
   GROUP BY "s"."course" ORDER BY "total" DESC
```

getMany()	→ Student entities (mapped objects)
getOne()	→ one Student or null
getRawMany()	→ plain rows, needed for COUNT/SUM/aliases that aren't entity columns

:name parameters are converted to $1, $2 — never put values inside the string itself.

SECTION 7 — LEVEL 3: RAW SQL (you write every character)

```
// through the repository
await this.studentRepo.query(
  'SELECT * FROM students WHERE age > $1 ORDER BY name',
  [20],
);

// or through the DataSource (any table, not tied to one entity)
constructor(private dataSource: DataSource) {}
await this.dataSource.query('SELECT COUNT(*) FROM students');
```

Returns plain objects (no entity mapping, no @UpdateDateColumn magic). Use for: Postgres-specific features (window functions, CTEs, JSONB operators), reports, performance tuning.

The SAME rule applies — parameters, never concatenation:

BAD	query(`SELECT * FROM students WHERE email = '${email}'`) email = "x' OR '1'='1" → returns every student (SQL injection)
GOOD	query('SELECT * FROM students WHERE email = $1', [email]) the value is sent separately; Postgres never treats it as SQL

SECTION 8 — TRANSACTIONS (several queries, all-or-nothing)

```
  await this.dataSource.transaction(async (manager) => {
    await manager.save(Student, s1);
    await manager.save(Student, s2);
    // if anything throws here -> ROLLBACK, neither row is saved
  });

SQL produced:  BEGIN; INSERT ...; INSERT ...; COMMIT;   (or ROLLBACK)
Use it whenever two writes must succeed or fail together.
```

SECTION 9 — WHICH LEVEL TO USE

Simple CRUD, filters, sorting, paging	→ repository + find options	(Section 5)
Joins, GROUP BY, aggregates, dynamic	→ QueryBuilder	(Section 6)
Postgres-only features, reports, tuning	→ raw .query()	(Section 7)
Several writes that belong together	→ transaction	(Section 8)

```
Comparison with Mongo (student.service.ts):
  Mongoose: studentModel.find()       -> MongoDB query object { }
  TypeORM:  studentRepo.find()        -> SQL text  SELECT ... FROM students
Same idea: the library turns method calls into the database's own
query language. Mongo's language is JSON-like objects; Postgres'
language is SQL.
```

★ RULE OF THUMB

You didn't write SQL because TypeORM writes it: entity (= table map) + repository call (= intent) → SQL + $1 parameters → pg driver → Postgres. Turn on `logging: true` to watch it happen. Start with repository methods, move to QueryBuilder for complex queries, drop to raw .query() only when needed — and at every level, values go in parameters, never inside the SQL string.

See also: **theory/41-supabase-postgres-crud-full-steps.txt** (the build), **theory/42-ways-to-connect-postgres.txt** (raw pg / Prisma / Knex alternatives), **theory/37-sql-cheatsheet.txt** (the SQL itself), **theory/40-postgres-end-to-end.txt**